

Instructions

Solve all 10 problems below. The problems are divided into three levels of increasing complexity. For every problem, submit all five of the following:

- Input-Process-Output (IPO) Table
- Algorithm (numbered, step-by-step)
- Pseudocode (using START, INPUT, PROCESS, IF, ELSE, DISPLAY, END)
- Flowchart (correct standard symbols)
- Short explanation (1-2 sentences on the logic used)

Python code is not required at this stage. You are assessed only on your algorithm, pseudocode, and flowchart logic. Python implementation will be introduced in a later lesson.

Flowcharts may be created using Mermaid, diagrams.net, MS Word shapes, PowerPoint shapes, or hand-drawn and scanned/photographed, as long as correct

Assessment Problems

LEVEL 1: BASIC LINEAR PROBLEMS

Simple input, process, and output - no decisions.

Problem 1: Auditorium Seat Count

An auditorium's seating is arranged in equal rows and columns. Create a logic design that asks for the number of rows and the number of columns, computes the total number of seats, and displays the result.

Problem 2: Fence Around a Lot

A square lot needs to be fenced on all sides. Create a logic design that asks for the length of one side, computes the perimeter, and displays the result.

Problem 3: Jeepney Fare Computation

A jeepney charges a fixed rate for every kilometer traveled. Create a logic design that asks for the rate per kilometer and the distance traveled, computes the total fare, and displays the result.

Problem 4: Examination Average

A student has three examination scores. Create a logic design that asks for the three

LEVEL 2: MULTIPLE-INPUT COMPUTATION PROBLEMS

Several inputs combined through more than one arithmetic operation - still no decisions.

Problem 5: Online Shopping Cart Total

An online shopper adds three different items to a cart, each with its own price and quantity. Create a logic design that asks for the price and quantity of each of the three items, computes the total cost, and displays the result.

Problem 6: Total Travel Cost

A family trip involves a fare, a toll fee, and a fuel cost. Create a logic design that asks for the fare, the toll fee, and the fuel cost, computes the total travel cost, and displays the result.

Problem 7: Semester Standing Computation

A student's semester standing is based on a quiz average, an examination score, and a project score, weighted at 25%, 45%, and 30% respectively. Create a logic design that asks for the three scores, computes the semester standing, and displays the result

LEVEL 3: DECISION PROBLEMS

IF/ELSE logic producing two or three possible outputs.

Problem 8: Senior Citizen Discount Eligibility

A restaurant gives a senior citizen discount to customers who are 60 years old or above. Create a logic design that asks for a customer's age and displays "Eligible for Senior Discount" if the age is 60 or above, or "Not Eligible for Senior Discount" otherwise.

Problem 9: Elevation Reading Classification

A hiking app records elevation relative to sea level. Create a logic design that asks for an elevation reading and displays "Above Sea Level" if it is greater than 0, "Below Sea Level" if it is less than 0, or "At Sea Level" if it is exactly 0.

Problem 10: Bus Seat Number Check

Bus seats are numbered for an out-of-town field trip. Create a logic design that asks for a seat number and displays "Even" if the number is divisible by 2, or "Odd" otherwise.

Problem 1 – Auditorium Seat Count

1. IPO Table

Input	Process	Output
Number of rows, Number of columns	Total Seats = Rows × Columns	Total number of seats

2. Algorithm

1. Start.
2. Input the number of rows.
3. Input the number of columns.
4. Multiply rows by columns.
5. Store the result as total seats.
6. Display the total seats.
7. End.

3. Pseudocode

```
START
INPUT Rows
INPUT Columns
PROCESS TotalSeats = Rows * Columns
DISPLAY "Total Seats = ", TotalSeats
END
```



4. Flowchart (Mermaid)

```
graph TD
    Start([Start]) --> B[/Input Rows/]
    B --> C[/Input Columns/]
    C --> D[TotalSeats = Rows * Columns]
    D --> E[/Display Total Seats/]
    E --> End([End])
```

Mermaid



5. Short Explanation

The program accepts the number of rows and columns, multiplies them, and displays the total number of seats in the auditorium.

Problem 2 – Fence Around a Lot

1. IPO Table

Input	Process	Output
Length of one side	Perimeter = $4 \times \text{Side}$	Perimeter of the lot

2. Algorithm

1. Start.
2. Input the length of one side.
3. Multiply the side by 4.
4. Store the result as perimeter.
5. Display the perimeter.
6. End.

3. Pseudocode

```
START
INPUT Side
PROCESS Perimeter = 4 * Side
DISPLAY "Perimeter = ", Perimeter
END
```



4. Flowchart (Mermaid)

Mermaid



```
flowchart TD
A([Start]) --> B[/Input Side Length/]
B --> C[Perimeter = 4 * Side]
C --> D[/Display Perimeter/]
D --> E([End])
```

5. Short Explanation

The program asks for the length of one side of the square lot, computes the perimeter by multiplying it by four, and displays the result.

Problem 3 – Jeepney Fare Computation

1. IPO Table

Input	Process	Output
Rate per kilometer, Distance traveled	Fare = Rate × Distance	Total fare

2. Algorithm

1. Start.
2. Input the rate per kilometer.
3. Input the distance traveled.
4. Multiply the rate by the distance.
5. Store the result as total fare.
6. Display the total fare.
7. End.

3. Pseudocode

```
START
INPUT Rate
INPUT Distance
PROCESS Fare = Rate * Distance
DISPLAY "Total Fare = ", Fare
END
```

4. Flowchart (Mermaid)

```
graph TD
    A([Start]) --> B[/Input Rate per Kilometer/]
    B --> C[/Input Distance/]
    C --> D[Fare = Rate × Distance]
    D --> E[/Display Total Fare/]
    E --> F([End])
```

5. Short Explanation

The program asks for the fare rate per kilometer and the distance traveled. It multiplies these values to compute and display the total fare.

Input	Process	Output
Exam 1, Exam 2, Exam 3	Average = $(\text{Exam1} + \text{Exam2} + \text{Exam3}) \div 3$	Examination Average

2. Algorithm

1. Start.
2. Input Exam 1 score.
3. Input Exam 2 score.
4. Input Exam 3 score.
5. Add the three scores.
6. Divide the sum by 3.
7. Store the result as the average.
8. Display the examination average.
9. End.

3. Pseudocode

```
START
INPUT Exam1
INPUT Exam2
INPUT Exam3
PROCESS Average = (Exam1 + Exam2 + Exam3) / 3
DISPLAY "Examination Average = ", Average
END
```



4. Flowchart (Mermaid)

Mermaid



```
flowchart TD
A([Start]) --> B[/Input Exam 1/]
B --> C[/Input Exam 2/]
C --> D[/Input Exam 3/]
D --> E[Average = (Exam1 + Exam2 + Exam3) / 3]
E --> F[/Display Average/]
F --> G([End])
```

5. Short Explanation

The program asks for three examination scores, computes their average by adding them together and dividing by three, then displays the result.

Input	Process	Output
Price1, Qty1, Price2, Qty2, Price3, Qty3	Total = (Price1 × Qty1) + (Price2 × Qty2) + (Price3 × Qty3)	Total shopping cost

2. Algorithm

1. Start.
2. Input the price and quantity of Item 1.
3. Input the price and quantity of Item 2.
4. Input the price and quantity of Item 3.
5. Compute the cost of each item by multiplying its price by its quantity.
6. Add the three item costs.
7. Store the result as the total cost.
8. Display the total shopping cost.
9. End.

3. Pseudocode

```
START
INPUT Price1, Qty1
INPUT Price2, Qty2
INPUT Price3, Qty3
PROCESS Cost1 = Price1 * Qty1
PROCESS Cost2 = Price2 * Qty2
PROCESS Cost3 = Price3 * Qty3
PROCESS Total = Cost1 + Cost2 + Cost3
DISPLAY "Total Cost = ", Total
END
```

4. Flowchart (Mermaid)

 Mermaid







```
flowchart TD
A([Start]) --> B[/Input Price1, Qty1/]
B --> C[/Input Price2, Qty2/]
C --> D[/Input Price3, Qty3/]
D --> E[Cost1 = Price1 × Qty1]
E --> F[Cost2 = Price2 × Qty2]
F --> G[Cost3 = Price3 × Qty3]
G --> H[Total = Cost1 + Cost2 + Cost3]
H --> I[/Display Total Cost/]
I --> J([End])
```

5. Short Explanation

The program calculates the cost of each item by multiplying its price by its quantity. It then adds all three costs to get the total amount to be paid.

Fare, Toll Fee, Fuel Cost

Total Cost = Fare + Toll Fee +
Fuel Cost

Total travel cost

2. Algorithm

1. Start.
2. Input the fare.
3. Input the toll fee.
4. Input the fuel cost.
5. Add the three costs.
6. Store the result as the total travel cost.
7. Display the total travel cost.
8. End.

3. Pseudocode

```
START
INPUT Fare
INPUT TollFee
INPUT FuelCost
PROCESS TotalCost = Fare + TollFee + FuelCost
DISPLAY "Total Travel Cost = ", TotalCost
END
```

4. Flowchart (Mermaid)

 Mermaid

```
graph TD
    A([Start]) --> B[/Input Fare/]
    B --> C[/Input Toll Fee/]
    C --> D[/Input Fuel Cost/]
    D --> E[TotalCost = Fare + Toll Fee + Fuel Cost]
    E --> F[/Display Total Travel Cost/]
    F --> G([End])
```

5. Short Explanation

The program accepts the fare, toll fee, and fuel cost as inputs. It adds these values together to compute and display the total travel cost.

Quiz Average, Examination Score, Project Score	Semester Standing = (Quiz × 25%) + (Exam × 45%) + (Project × 30%)	Semester Standing
---	--	-------------------

2. Algorithm

1. Start.
2. Input the quiz average.
3. Input the examination score.
4. Input the project score.
5. Multiply the quiz average by 25%.
6. Multiply the examination score by 45%.
7. Multiply the project score by 30%.
8. Add the weighted scores.
9. Store the result as the semester standing.
10. Display the semester standing.
11. End.

3. Pseudocode

```
START
INPUT QuizAverage
INPUT ExamScore
INPUT ProjectScore
PROCESS SemesterStanding = (QuizAverage * 0.25) + (ExamScore * 0.45) + (ProjectScore * 0.30)
DISPLAY "Semester Standing = ", SemesterStanding
END
```

4. Flowchart (Mermaid)

```
graph TD
    Start([Start]) --> B[/Input Quiz Average/]
    B --> C[/Input Examination Score/]
    C --> D[/Input Project Score/]
    D --> E[SemesterStanding = (Quiz × 0.25) + (Exam × 0.45) + (Project × 0.30)]
    E --> F[/Display Semester Standing/]
    F --> End([End])
```

5. Short Explanation

The program computes the student's semester standing using the given weighted percentages for the quiz, examination, and project scores, then displays the final result.

1. IPO Table

Input	Process	Output
Age	Check if Age $\geq$ 60	"Eligible for Senior Discount" or "Not Eligible for Senior Discount"

2. Algorithm

1. Start.
2. Input the customer's age.
3. Check if the age is 60 or above.
4. If true, display "Eligible for Senior Discount."
5. Otherwise, display "Not Eligible for Senior Discount."
6. End.

3. Pseudocode

```
START
INPUT Age
IF Age >= 60 THEN
    DISPLAY "Eligible for Senior Discount"
ELSE
    DISPLAY "Not Eligible for Senior Discount"
END IF
END
```

4. Flowchart (Mermaid)

Mermaid

```
graph TD
    A([Start]) --> B[/Input Age/]
    B --> C{Age ≥ 60?}
    C -- Yes --> D[/Display "Eligible for Senior Discount"/]
    C -- No --> E[/Display "Not Eligible for Senior Discount"/]
    D --> F([End])
    E --> F
```

5. Short Explanation

The program checks whether the customer's age is at least 60. It displays the appropriate message based on the result of the condition

Input	Process	Output
Elevation	Compare elevation with 0	"Above Sea Level", "Below Sea Level", or "At Sea Level"

2. Algorithm

1. Start.
2. Input the elevation reading.
3. If the elevation is greater than 0, display "Above Sea Level."
4. Else if the elevation is less than 0, display "Below Sea Level."
5. Otherwise, display "At Sea Level."
6. End.

3. Pseudocode

```
START
INPUT Elevation
IF Elevation > 0 THEN
    DISPLAY "Above Sea Level"
ELSE
    IF Elevation < 0 THEN
        DISPLAY "Below Sea Level"
    ELSE
        DISPLAY "At Sea Level"
    END IF
END IF
END
```



4. Flowchart (Mermaid)

```
graph TD
    A([Start]) --> B[/Input Elevation/]
    B --> C{Elevation > 0?}
    C -- Yes --> D[/Display "Above Sea Level"/]
    C -- No --> E{Elevation < 0?}
    E -- Yes --> F[/Display "Below Sea Level"/]
    E -- No --> G[/Display "At Sea Level"/]
    D --> H([End])
    F --> H
    G --> H
```



5. Short Explanation

The program compares the elevation with zero. It determines whether the location is above sea level, below sea level, or exactly at sea level.

1. IPO Table

Input	Process	Output
Seat Number	Check if Seat Number is divisible by 2	"Even" or "Odd"

2. Algorithm

1. Start.
2. Input the seat number.
3. Check if the seat number is divisible by 2.
4. If true, display "Even."
5. Otherwise, display "Odd."
6. End.

3. Pseudocode

```
START
INPUT SeatNumber
IF SeatNumber MOD 2 = 0 THEN
    DISPLAY "Even"
ELSE
    DISPLAY "Odd"
END IF
END
```

4. Flowchart (Mermaid)

```
graph TD
    Start([Start]) --> B[/Input Seat Number/]
    B --> C{Seat Number MOD 2 = 0?}
    C -- Yes --> D[/Display "Even"/]
    C -- No --> E[/Display "Odd"/]
    D --> F([End])
    E --> F
```

5. Short Explanation

The program uses the modulus (MOD) operator to determine whether the seat number is divisible by 2. If the remainder is 0, the number is even; otherwise, it is odd.

Python Basics

1. Print Function
print("Hello, Python!")

2. Variables
name = "Alice"
age = 20
print("Name:", name)
print("Age:", age)

3. Data Types
is_student = True # Boolean
height = 5.5 # Float
count = 10 # Integer
name = "Bob" # String

4. Basic Operations

a = 10	
b = 3	+ Add
print("Add:", a + b)	- Subtract
print("Subtract:", a - b)	* Multiply
print("Multiply:", a * b)	/ Divide
print("Divide:", a / b)	// Floor Division
print("Floor Division:", a // b)	% Modulus (Remainder)
print("Modulus:", a % b)	** Power (Exponent)
print("Power:", a ** b)	

5. List

```
fruits = ["apple", "banana", "cherry"]  
print("Fruits:", fruits)  
print("First fruit:", fruits[0])
```


Python Basics - Part 2

6. Conditionals

```
age = 18
if age >= 18:
    print("You are an adult.")
elif age >= 13:
    print("You are a teenager.")
else:
    print("You are a child.")
```

Comparison Operators

```
== Equal to
!= Not equal to
> Greater than
< Less than
>= Greater than or equal to
<= Less than or equal to
```

2 / 2

7. Loops

```
# For loop
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

```
# While loop
count = 1
while count <= 5:
    print("Count:", count)
    count += 1
```

8. Functions

```
def greet(name):
    return f"Hello, {name}!"
message = greet("Python")
print(message)
```

```
# def = define a function
# return = send a value back
# to the place where the
# function was called
```

9. Lists (More)

```
numbers = [1, 2, 3, 4, 5]
numbers.append(6)      # Add to end
numbers.insert(0, 0)   # Insert at index 0
numbers.remove(3)      # Remove first occurrence of 3
numbers.pop()          # Remove last item
print(numbers)
```

```
# Common List Methods
.append(x) → Add at end
.insert(i, x) → Insert at index i
.remove(x) → Remove item x
.pop() → Remove & return last item
.clear() → Remove all items
```

10. Dictionaries

```
person = {"name": "Alice", "age": 20, "city": "New York"} # Key: Value pairs
print(person["name"]) # Access value # Keys must be unique
person["age"] = 21 # Update value # and immutable
person["email"] = "alice@example.com" # Add new key-value pair (like strings or numbers)
del person["city"] # Delete key-value pair
print(person)
```



PYTHON BASICS

The building blocks of Python 🚀

1 PRINT()

Used to output data to the screen.

```
# Printing text
print("Hello, Python!")
```

2 VARIABLES

Used to store data values.

```
# Variables
name = "Python"
age = 25
is_fun = True
```

3 DATA TYPES

Different types of data in Python.

```
# Data Types
text = "Hello"      # str
num = 10            # int
price = 9.99        # float
is_valid = False    # bool
```

4 INPUT()

Used to take input from the user.

```
# User Input
name = input("Enter your name: ")
print("Hi, " + name + "!")
```

5 CONDITIONALS

Used to make decisions.

```
# If-Else
age = 18
if age >= 18:
    print("You are an adult.")
else:
    print("You are a minor.")
```

6 LOOPS

Used to repeat a block of code.

```
# For Loop
for i in range(5):
    print("Count:", i)
```

7 LISTS

Used to store multiple items in a single variable.

```
# List
fruits = ["apple", "banana", "cherry"]
print(fruits[0]) # Output: apple
```



Practice daily. Code. Learn. Repeat. 🚀